

Compositional Action Models

Architecture & Redesign Analysis

Component-level map · interface schemas · migration path
for the compositional / zero-shot research direction

Generated 2026-08-15 · branch: multiskill

Compositional Action Models — Architecture & Redesign Analysis

A component-level map of the current system, the design decisions baked into it, and the interface changes required to support the compositional / zero-shot research direction in a modular and interpretable way.

0. How to read this document

This is a **redesign brief**, not a tutorial. Each subsystem is described three ways:

1. **Responsibility** — the single job it owns.
2. **Interface / schema** — the contract it exposes today (inputs → outputs).
3. **Rigidity & redesign needs** — where the current contract blocks the research direction, and the interface it should expose instead.

The goal is that any one component can be understood, replaced, or re-specified *in isolation*. Section 3 is the current system. Section 5 is the honest list of what will hurt to change. Section 6 is the proposed target architecture, written as interfaces so it can be adopted piecemeal.

Legend for severity/effort tags: ● low · ● medium · ● high.

1. Research thesis & direction

1.1 Core hypothesis

*A single low-level continuous-control policy can perform many manipulation skills, where the **only** signal distinguishing one skill from another is a vector **embedding of that skill's symbolic action model** (a PDDL-style operator). If that embedding carries real semantic/compositional structure, the shared policy should (a) learn faster than from an arbitrary skill id and (b) **generalize to skills it never trained on**, by composing an embedding for a novel operator out of parts it has seen.*

1.2 The direction ladder

The repository today sits on rung 2. The name ("compositional action models") and the placeholder fields in the code point at rungs 3–4.

Rung	Capability	Status
1	One policy per skill, embedding injected as a constant observation feature	✓ <code>train_skill.py</code>
2	One shared policy, several skills, embedding (frozen or trainable) as the sole selector	✓ <code>train_multiskill.py</code>
3	Embedding composed from operator parts (predicates, params, effects) via learnable component-embedders + aggregator	✗ placeholders only
4	Zero-shot execution of a held-out operator by composing its embedding at eval time	✗ interface does not allow it
5	Interpretable embedding space (component ablation, arithmetic, probing) drives claims	⚠ only PCA-of-table today

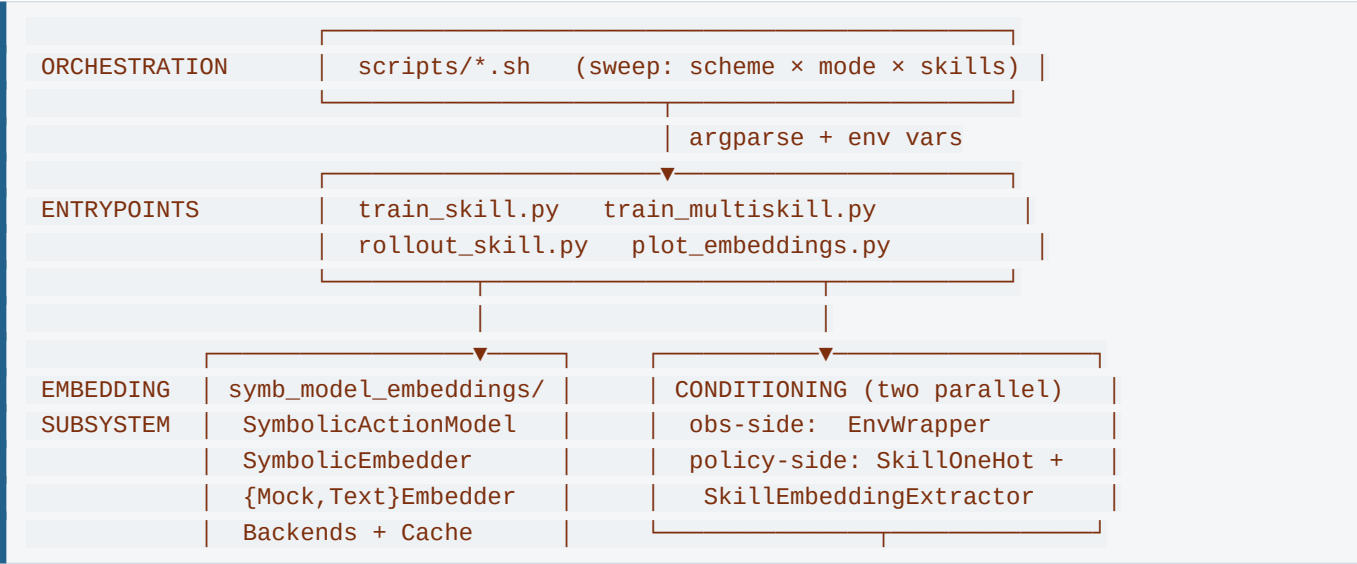
1.3 What "supporting the research direction" concretely requires

Four capabilities the current architecture cannot express without interface changes (detailed in §5–§6):

- **C1 — Structured operators.** A parsed operator (typed params, predicate lists for pre/effects), not a free-text string.
- **C2 — Composable embedding.** `operator` → `components` → `aggregate` → `vector`, end-to-end differentiable when desired.
- **C3 — Per-episode embedding by value, not by table index.** So a *novel* operator's freshly-composed embedding can condition the policy with no reserved table row.
- **C4 — An evaluation + interpretability harness** that measures held-out-skill success and probes what the embedding encodes.

2. System at a glance

2.1 Layered architecture



```

ENVIRONMENT (fetch_blockworld) | FetchSkillEnv — make_skill_env
                                | FactEvaluator (obs → predicates)
                                | reward shapers + skill_reward dispatcher
                                | multiblock_env (runtime MJCF synthesis)
                                | scripted teachers + BC warm-start

```

| Gymnasium-Robotics / MuJoCo

2.2 Episode lifecycle (multi-skill path)

```

reset → MultiSkillEnv picks a skill
      → that skill's FetchSkillEnv.reset() (putdown: runs scripted pickup first)
      → FactEvaluator.reset_reference(obs) (remember table-z / initial poses)
      → SkillOneHotWrapper appends skill_onehot to obs
step  → policy(obs) → action [dx,dy,dz,grip]
      → FetchSkillEnv.step: skill_reward(name, obs, evaluator, action, shapers)
                                → reward, success, facts, components
      → terminate_on_success flips `terminated`
policy conditioning:
      SkillEmbeddingExtractor: features = concat(flatten(obs\skill_onehot),
                                                onehot @ embedding_table)

```

2.3 Module inventory

Path	Role	LOC-ish	Redesign pressure
fetch_blockworld/facts.py	obs → symbolic predicates	280	● obs indices hardcoded
fetch_blockworld/skills.py	skill registry + reward dispatcher	310	● dispatcher & registry entangled
fetch_blockworld/rewards/*.py	5 bespoke shapers	~600	● per-skill, non-uniform scale
fetch_blockworld/envs.py	FetchSkillEnv, make_skill_env	170	● shaper wiring hardcoded
fetch_blockworld/multiblock_env.py	runtime multi-block MJCF	305	● self-contained
fetch_blockworld/scripted.py	scripted controllers/teachers	388	● per-skill, no novel-skill path
fetch_blockworld/teacher.py	BC warm-start	140	● generic-ish
symb_model_embeddings/action_model.py	operator value object	50	● unstructured; placeholders unused
symb_model_embeddings/embedders.py	embedder interface + impls +	206	● trainable

Path	Role	LOC-ish	Redesign pressure
	CLI		overloaded/broken
<code>symb_model_embeddings/backends.py</code>	text backends	141	● clean
<code>symb_model_embeddings/cache.py</code>	disk embedding cache	90	● clean
<code>symb_model_embeddings/trainable_embedding.py</code>	policy-side table extractor	111	● table-indexed, fixed N
<code>symb_model_embeddings/obs_symb_wrapper.py</code>	obs-side embedding inject	38	● hardcoded shape (25,)
<code>symb_model_embeddings/naming.py</code>	run/file naming	59	● fine
<code>symb_model_embeddings/mock_env_embedder.py</code>	dead legacy	11	● delete
<code>train_multiskill.py</code>	multi-skill driver	417	● owns too much glue
<code>plot_embeddings.py</code>	PCA of embedding trajectories	152	● only interpretability today

3. Component reference (current system)

Each entry: **Responsibility** · **Interface** · **Design decisions** · **Rigidity / redesign needs**.

3.1 **FactEvaluator** — symbolic state extractor

Responsibility. Turn a raw Fetch observation into (a) a structured **FetchState** and (b) a dict of boolean predicates, with all thresholds centralized for tuning.

Interface (today).

```
FactEvaluator(num_blocks=1, *, object_half_size, near_tol, xy_tol, z_tol,
              goal_tol, push_distance, lift_height, open_width, closed_width,
              stack_xy_tol, stack_clear_dist)

.reset_reference(obs) -> None          # snapshot table-z / initial block poses
.state(obs)          -> FetchState    # numeric decode
.facts(obs)          -> dict[str,bool]# predicates (see §4.2)
.is_true(obs, name)  -> bool
.numeric_summary(obs) -> dict         # for logging/info
```

Design decisions. - Thresholds live in one place, deliberately decoupled from RL code — good. - Reference

poses (`table_object_z`, `initial_object_pos`) are captured at reset so predicates like `object_on_table` judge against the *randomized resting* pose, not the current one.

Rigidity / redesign needs. 🟡 - **Observation layout is hardcoded by slice:** `grip=0:3, obj=3:6, fingers=9:11, rot=11:14, has_object = shape>=25`, multi-block appended at `25 + 9*(i-1)`, one-hots trailing. Any env/layout change silently corrupts facts. → Needs an explicit **ObservationSchema** (§6.8) that names fields so fact code never indexes raw arrays. - The **predicate vocabulary is a flat dict of hand-named booleans**. The compositional embedder (§6.2) will want predicate *symbols* that line up with the operator's precondition/effect predicates. Today the fact names (`object_lifted`) and the operator predicates (`holding ?o`) are **different vocabularies** with no mapping — a gap that must close for grounding-aware work.

3.2 SkillSpec registry + skill_reward dispatcher

Responsibility. Define each skill (env id, success predicate, step budget, PDDL string) and compute its shaped reward.

Interface (today).

```
@dataclass(frozen=True)
class SkillSpec:
    name: str
    env_id: str
    success_fact: str
    max_episode_steps: int = 75
    terminate_on_success: bool = True
    action_model: str | None = None      # free-text PDDL operator

SKILLS: dict[str, SkillSpec]           # 9 skills
require_skill(name) -> SkillSpec

skill_reward(name, obs, evaluator, *, action, reward_config,
             pickup_reward_shaper, putdown_reward_shaper, push_reward_shaper,
             stack_reward_shaper, unstack_reward_shaper)
    -> (reward: float, success: bool, facts: dict, components: dict)
```

Design decisions. - Symbolic description travels *with* the skill (`action_model` on `SkillSpec`) — the one place symbolic and control meet today. - Reward components are returned for logging/interpretability.

Rigidity / redesign needs. 🔴 **This is the highest-coupling file.** - `skill_reward` is a **string-keyed if/elif ladder** that takes *one keyword argument per shaper*. Adding a skill edits this signature, the ladder, the `SKILLS` dict, `envs.py` shaper wiring, `FactEvaluator`, and `scripted.py` — ~5 coordinated edits. → Needs a uniform **RewardModule interface + a single RewardContext** (§6.7) so skills register a reward object, not a branch. - `action_model` is an **opaque string** — see §3.7.

3.3 Reward shapers (rewards/*.py)

Responsibility. Per-skill dense, staged, stateful reward (e.g. pickup has 7 stages: reach → above → descend → grip → off-table → lift → success, each with one-time bonuses and progress terms clipped against noise).

Interface (today). Each shaper is a `@dataclass` with `reset()` and a `compute(**skill_specific_kwargs) -> (reward, components[, success])`. The signatures differ per shaper (pickup takes `grip_pos`, `object_pos`, `lift`, ...; stack takes `mover_pos`, `base_pos`, ...).

Rigidity / redesign needs. ● - Non-uniform `compute` signatures are why the dispatcher needs per-shaper kwargs. → Standardize on `compute(ctx: RewardContext) -> RewardResult`. - **Reward scales differ wildly** across skills (pickup peaks ~+15 success bonus + staged bonuses; `reach_top` is `-||grip-top|| + 2`; push has its own). In multi-skill training the **shared value function sees inconsistent return magnitudes per skill** — a genuine *confound* for the very ablation the project runs (does semantic embedding beat mock?). → §6.7 proposes a normalization / reward-scale contract. - Shapers are stateful and reset from `FetchSkillEnv.reset` — fine, but the ownership is implicit.

3.4 FetchSkillEnv + make_skill_env

Responsibility. Wrap a native/(multi-block) Fetch env into a single-skill task: swap reward/termination, expose facts in `info`, prepare non-trivial reset states (putdown starts from a scripted grasp).

Interface (today).

```
make_skill_env(skill_name, render_mode=None, seed=None) -> FetchSkillEnv
FetchSkillEnv(env, skill: SkillSpec, evaluator=None)
    .reset() -> (obs, info{facts, numeric_state, skill, [scripted_pickup_*]})
    .step(a) -> (obs, reward, terminated, truncated,
                info{is_success, facts, numeric_state, skill, reward_components})
```

Rigidity / redesign needs. ● - The constructor **instantiates all five shapers by `if skill.name == ...`** → Should ask a `RewardModule` factory (§6.7). - Otherwise a clean Gymnasium `Wrapper`; the `info` schema (§4.4) is a good, stable surface to build eval/interpretability on.

3.5 multiblock_env.py — runtime multi-block scene

Responsibility. Provide stack/unstack scenes (Gymnasium-Robotics ships only single-block Fetch) by **synthesizing an MJCF at runtime** that `<include>`s the stock assets by absolute path, then registers `FetchStackEnv-v0` / `FetchUnstackEnv-v0`.

Interface (today).

```

build_multiblock_xml(num_blocks, assets_dir=None) -> str
MujocoFetchMultiBlockEnv(num_blocks=2, start_stacked=False, reward_type="sparse")
# obs layout: [0:25 standard] + per-extra-block[pos3,rel3,rot3] + mover_onehot(N) +
base_onehot(N)
# each reset samples mover_index != base_index, appended as one-hots

```

Rigidity / redesign needs. ● Well-encapsulated and self-contained (no site-packages edits). The **only** coupling worth noting: the observation layout it emits is the same one `FactEvaluator` decodes by hand — both should read a shared `ObservationSchema` (§6.8) so they can never drift.

3.6 Scripted controllers + BC (`scripted.py`, `teacher.py`)

Responsibility. (a) Reset preparation (`run_scripted_pickup`). (b) A stateless per-skill teacher `scripted_teacher_action(name, obs, evaluator)` used to collect demonstrations. (c) `behavior_clone_policy` — supervised warm-start of the SB3 policy by MSE on teacher actions before RL.

Interface (today).

```

scripted_teacher_action(skill_name, obs, evaluator, config?) -> action[4]
collect_teacher_dataset(env, skill_name, config, *, seed) -> TeacherDataset
behavior_clone_policy(model, dataset, TeacherConfig) -> None
warm_start_with_teacher(model, env, skill_name, config, *, seed) -> None

```

Rigidity / redesign needs. ● - `scripted_teacher_action` is a **string-keyed dispatch** with per-skill helpers — same additive-cost problem as the reward ladder. - **Structural blocker for rung 4:** a *novel* (held-out) operator has **no scripted teacher** by definition, so any zero-shot claim must work *without* BC. Today hard skills lean on BC to succeed at all. The research plan needs an explicit answer: teacher only for *seen* skills; novel skills evaluated on RL / embedding transfer alone. - BC updates **all** policy params (including the embedding table when trainable) — worth being deliberate about whether BC should move embeddings.

3.7 `SymbolicActionModel` — the operator value object

Responsibility. The env-independent description of an operator that an embedder consumes.

Interface (today).

```

@dataclass(frozen=True)
class SymbolicActionModel:
    name: str
    description: str | None = None          # full PDDL string (free text)
    # --- declared but NEVER populated or read: ---
    types: tuple[str,...] = ()
    parameters: tuple[str,...] = ()

```



```

preconditions: tuple[str,...] = ()
effects: tuple[str,...] = ()
def text(self, source) -> str          # "name" | "action_model"

```

Rigidity / redesign needs. ● **This is the crux of the whole redesign.** - The compositional fields exist as **placeholders and are never filled** — every embedder today uses only `name` or the whole `description` string. So the system is "compositional" in name only. - There is **no parser** from the PDDL string to structured parts. Compositional embedding (rung 3) is impossible until `SymbolicActionModel` becomes a real **structured operator** (typed params + predicate lists) and a parser exists. → §6.1.

3.8 `SymbolicEmbedder` interface + `Mock/Text` + `EmbedderConfig`

Responsibility. Map a `SymbolicActionModel` to a fixed 1-D vector; expose a config + argparse surface for ablations.

Interface (today).

```

class SymbolicEmbedder(ABC):
    trainable: bool = False
    def embed(self, action_model) -> np.ndarray    # 1-D

MockEmbedder(size)          # sha256(name) → RNG → vector    (distinct, meaningless)
TextEmbedder(backend, source="name|"action_model")

@dataclass(frozen=True)
class EmbedderConfig:
    kind="mock|"text"; source; backend; model; size; trainable=False
    build_embedder(config, *, cache=None) -> SymbolicEmbedder

```

Rigidity / redesign needs. ● - `build_embedder(trainable=True)` raises `NotImplementedError`. Yet trainable embeddings are implemented — in `SkillEmbeddingExtractor`. So the word "trainable" has **two contradictory meanings**: (i) "produce a learnable embedder object" (unimplemented) vs (ii) "set `requires_grad` on the policy's table" (implemented). `train_multiskill.py` works around this by forcibly `replace(config, trainable=False)` when building initial values. → The redesign must **unify** these: an embedder pipeline that is *optionally* an `nn.Module` and can live either as frozen data or as a policy submodule (§6.5). - The `embed(action_model) -> np.ndarray` contract is **numpy, single-vector, stateless** — fine for frozen text/mock, but a *learnable compositional* embedder needs a torch, batched, differentiable path. The interface should admit both (§6.2–§6.5).

3.9 Text backends + cache

Responsibility. `TextEmbeddingBackend.embed_text(str) -> vec`, with `OpenAIBackend` (real) and `HashBackend` (offline, reproducible baseline), routed through a JSON `DiskEmbeddingCache` (atomic writes,

merge-on-flush).

Rigidity / redesign needs. ● Clean, well-abstracted, reusable as-is. One research note: the *only* semantically-meaningful signal comes from `openai`; the reproducible default (`hash`) is meaningless by design — so the headline "does semantics help?" result **requires API calls**, while CI-friendly runs can't answer it. Worth documenting as an experimental constraint, not a code defect.

3.10 Conditioning mechanism A — `EnvWrapper` (obs-side, frozen)

Responsibility. Inject a constant embedding vector into the observation dict under `symb_embedding` (single-skill path, `train_skill.py/rollout_skill.py`).

Interface (today).

```
EnvWrapper(env, symb_embedding: np.ndarray)
# observation_space = Dict(observation(25,), achieved_goal(3), desired_goal(3),
# symb_embedding(k,))
```

Rigidity / redesign needs. ● - **Hardcodes observation shape (25,)** → cannot wrap stack/unstack (multi-block obs are longer). So the obs-side path silently excludes the very two-parameter operators that the compositional story needs most. - It is **one of two** unrelated conditioning schemes (see 3.11). Two schemas, two code paths, no shared contract. → §6.5 replaces both.

3.11 Conditioning mechanism B — `SkillOneHotWrapper` + `SkillEmbeddingExtractor`

Responsibility. Multi-skill conditioning: put a **one-hot skill id** in the obs; keep the **embedding table inside the policy** as an `nn.Parameter`; select a row by `onehot @ table` (so gradients flow when trainable).

Interface (today).

```
SkillOneHotWrapper(env, skill_index, num_skills)      # obs["skill_onehot"] = e_i
SkillEmbeddingExtractor(obs_space, init_embeddings[N,d], trainable)
    forward(obs) -> concat( flatten(obs \ skill_onehot), onehot @ table )
    current_embeddings() -> np.ndarray                # for logging
```

Rigidity / redesign needs. ● **This is the wall in front of rung 4.** - Conditioning is **by table index**, and the table has a **fixed N = num_skills fixed at construction**. There is **no way to condition on an operator that has no reserved row** → **zero-shot is structurally impossible** here. - The one-hot encodes *identity*, deliberately discarding structure — which is the right *baseline* but the wrong *general mechanism*. → The policy must be able to accept a **conditioning vector by value**, computed per-episode from the operator by the embedding pipeline, so a novel operator's vector works with no retraining (§6.5, C3).

3.12 Entrypoints & orchestration

- `train_skill.py` (rung 1), `train_multiskill.py` (rung 2), `rollout_skill.py` (eval/render, single-skill only), `scripts/train_all_methods.sh` (scheme × mode sweep), `plot_embeddings.py` (PCA of embedding trajectories).
- `naming.py` gives every run a filename encoding skill/algo/embed-tag/teacher/seed so runs never overwrite — good, but it is also **the only "experiment config" that exists**; there is no single serializable spec (§6.9).

Rigidity / redesign needs. 🟡 `train_multiskill.py` (417 LOC) owns env construction, initial-embedding building, BC pooling, extractor discovery, logging *and* the training loop. It should orchestrate small components, not implement them.

4. Cross-cutting schemas (consolidated)

4.1 Observation vector layout (single + multi-block)

index	field	notes
0:3	gripper position	
3:6	object0 position	present iff len(obs) >= 25 (has_object)
6:9	object0 relative-to-grip	
9:11	gripper finger state	gripper_width = sum(9:11)
11:14	object0 rotation (euler)	
14:25	velocities / misc (standard Fetch)	
-- multi-block only (num_blocks N > 1): --		
25 + 9*(i-1) + [0:3, 3:6, 6:9]	block_i pos, rel, rot	for i in 1..N-1
onehot_base + [0:N]	mover one-hot	
onehot_base + [N:2N]	base one-hot	onehot_base = 25 + 9*(N-1)

Consumed by `FactEvaluator.state()` via literal indices; produced by `multiblock_env._get_obs()`. These two must agree and currently do so only by convention — see §6.8.

4.2 Fact predicate schema (single-block core)

`has_object`, `object_on_table`, `object_lifted`, `gripper_near_object`, `gripper_above_object`, `gripper_touching_object_top`, `gripper_open`, `gripper_closed`, `holding_object`, `object_at_goal`, `object_moved_{left,right,forward,backward}`

Multi-block adds: `has_two_blocks`, `mover_lifted`, `mover_on_table`, `gripper_near_mover`, `mover_held`, `blocks_stacked`, `mover_clear_of_base`.

4.3 Symbolic action-model schema

Today: `{name, description(PDDL string)}` used; `{types, parameters, preconditions, effects}` declared-but-empty. **Proposed structured form:** §6.1.

4.4 info dict schema (stable surface for eval/interpretability)

```
reset: facts, numeric_state, skill, [scripted_pickup_success, scripted_pickup_steps]
step : is_success(float), facts, numeric_state, skill, reward_components(dict)
multi: + info["skill"] set by MultiSkillEnv each reset
```

4.5 Embedding contract (today, fractured)

Path	Where embedding lives	Keyed by	Trainable?	Multi-block?
EnvWrapper	obs["symb_embedding"] (data)	precomputed vector	no	✗ (shape 25 hardcoded)
SkillEmbeddingExtractor	policy nn.Parameter table	one-hot index	yes/no	✓

One contract should replace both — §6.5.

5. Design tensions & rigidity register

The blocking issues, ranked by how much they impede the research direction. "Blast radius" = files that must change together.

#	Issue	Why it blocks research	Blast radius	Sev
1	No zero-shot conditioning path — policy conditions by fixed-N table index	Rung 4 impossible; can't feed a novel operator's vector	trainable_embedding.py, train_multiskill.py, policy	●
2	Operator is an opaque string — structured fields unused, no parser	Rung 3 (compositional embedding) has no substrate	action_model.py, embedders	●
3	Two unrelated conditioning schemes (obs-side vs policy-side), no shared contract	Every new capability must be built twice / chooses one silo	obs_symb_wrapper.py, trainable_embedding.py, both trainers	●
4	trainable overloaded; build_embedder(trainable=True) raises while trainability lives elsewhere	Learnable <i>compositional</i> embedder has no home in the embedder API	embedders.py, train_multiskill.py	●
5	Reward dispatcher is a string ladder w/ per-shaper kwargs	Adding/holding-out skills is ~5 coordinated edits; error-prone	skills.py, envs.py, rewards/*, scripted.py	●
6	Non-uniform reward scale across skills	Confounds the shared-policy ablation (semantic vs mock)	rewards/*, skills.py	●
7	Obs layout hardcoded by slice in fact	Fragile to any env/space change;	facts.py, multiblock_env.py	●

#	Issue	Why it blocks research	Blast radius	Sev
	code	blocks clean var-block support		
8	EnvWrapper hardcodes obs shape (25,)	Single-skill embedding path can't run stack/unstack	obs_symb_wrapper.py	●
9	No teacher for novel skills (scripted per-skill)	Zero-shot must run without BC; current hard skills depend on it	scripted.py, teacher.py	●
10	No evaluation harness / no committed metrics (models/ empty)	Can't substantiate any research claim	— (missing)	●
11	Interpretability = PCA of table only	Rung 5 needs ablation/arithmetic/probing	plot_embeddings.py	●
12	No single serializable experiment config — spread across argparse + env vars + 3 dataclasses; filename is the record	Reproducibility & sweep management fragile	entrypoints, scripts/*, naming.py	●
13	Fact vocab ≠ operator predicate vocab	Grounding-aware / component-to-predicate work has no mapping	facts.py, action_model.py	●
14	Dead code mock_env_embedder.py (torch, stale skill names)	Confuses the embedding story	1 file	●

6. Target architecture (proposed interfaces)

Design principles: **(P1)** one contract per concept, no parallel silos; **(P2)** everything the research varies is a swappable component behind a small interface; **(P3)** the same embedding pipeline works frozen or learnable and *by value* (so novel operators need no special case); **(P4)** interpretability is a first-class output, not an afterthought.

Each interface below is written so it can be adopted independently.

6.1 StructuredOperator + PDDL parser (fixes #2, #13)

Replace the opaque string with a parsed structure; keep the string as the parser input.

```
@dataclass(frozen=True)
class Predicate:
    name: str          # e.g. "on", "clear", "holding"
    args: tuple[str, ...] # variable names, e.g. ("?o", "?b")
```

```

@dataclass(frozen=True)
class Param:
    var: str          # "?o"
    type: str         # "block"

@dataclass(frozen=True)
class Effect:
    predicate: Predicate
    add: bool         # True = add, False = delete ((not ...))

@dataclass(frozen=True)
class StructuredOperator:
    name: str
    params: tuple[Param, ...]
    preconditions: tuple[Predicate, ...]
    effects: tuple[Effect, ...]
    raw: str | None = None # original PDDL, for text-embedder fallback

def parse_pddl_operator(text: str) -> StructuredOperator: ...
def operator_from_spec(spec: SkillSpec) -> StructuredOperator: ...

```

`SymbolicActionModel` becomes a thin adapter over `StructuredOperator` (or is replaced by it). All existing PDDL strings in `SKILLS` already parse into this.

6.2 `ComponentEmbedder` — embeds atomic pieces (*enables rung 3*)

```

class ComponentEmbedder(Protocol):
    dim: int
    def embed_predicate(self, p: Predicate) -> Tensor: ...
    def embed_param(self, p: Param) -> Tensor: ...
    def embed_type(self, t: str) -> Tensor: ...

```

Two natural implementations, both behind the same Protocol: - `LearnableComponentEmbedder` — `nn.Embedding` over a symbol vocabulary (predicate names, types, arg-roles). Differentiable; the primary research object. - `TextComponentEmbedder` — reuse §3.9 backends to embed each symbol's text (frozen; strong baseline; needs no vocabulary).

6.3 `Aggregator` — components → one operator vector (*enables rung 3*)

```

class Aggregator(nn.Module):
    out_dim: int
    def forward(self, op: StructuredOperator,
                emb: ComponentEmbedder) -> Tensor: ... # (out_dim,) or (B,out_dim)

```

Interchangeable implementations (this is the core ablation axis for the thesis): - `MeanPoolAggregator` (order-invariant baseline), - `DeepSetsAggregator` (learned permutation-invariant), - `AttentionAggregator` / small transformer over the component set, - `GraphAggregator` (GNN over the predicate–argument graph — captures which predicates share variables).

6.4 EmbeddingPipeline — the composition (unifies #4)

```
class EmbeddingPipeline(Protocol):
    dim: int
    trainable: bool
    def embed(self, op: StructuredOperator) -> Tensor: ... # torch, batchable
    def as_numpy(self, op) -> np.ndarray: ... # frozen convenience
```

Concrete: `CompositionalPipeline(component_embedder, aggregator)` and, for baselines, `WholeStringPipeline(text_backend)` and `MockPipeline` — all the same interface. This **single interface replaces the split** between `build_embedder` (frozen numpy) and the extractor's table (learnable torch): frozen pipelines expose `.as_numpy`; learnable ones are `nn.Modules` dropped into the policy (§6.5). Retire `EmbedderConfig.trainable`'s double meaning.

6.5 ConditioningModule — one way the embedding reaches the policy (fixes #1, #3, #8)

The keystone. The env emits an **operator handle** (and/or a precomputed vector) in the observation; a single policy-side module turns it into the conditioning vector, so *how it's produced* (table / text / compositional) and *whether it's learnable* are internal details — and **novel operators need no table row**.

```
# Observation carries the operator, by value or by reference:
obs["operator_id"] : int # index into a known operator set (optional)
obs["operator_embed"]: float[dim] # precomputed vector (frozen path)
# (For learnable/compositional, the env passes an operator handle the module
# can embed; for fixed sets, an id suffices.)

class ConditioningModule(BaseFeaturesExtractor):
    """features = concat(flatten(base_obs), conditioning_vector)."""
    def __init__(self, obs_space, pipeline: EmbeddingPipeline | None,
                 embedding_table: Optional[Tensor], mode: Literal[
                     "table", "by_value", "compositional"]): ...
    def forward(self, obs) -> Tensor: ...
    def current_embeddings(self) -> np.ndarray: ... # keep the logging hook
```

- `mode="table"` reproduces today's `SkillEmbeddingExtractor` (baseline).
- `mode="by_value"` reads `obs["operator_embed"]` → **frozen zero-shot works**.
- `mode="compositional"` runs the learnable `EmbeddingPipeline` on the operator handle → **learnable zero-shot**, gradients into components/aggregator.

This one module makes `EnvWrapper` and `SkillOneHotWrapper+SkillEmbeddingExtractor` two *configs* of the same thing instead of two subsystems.

6.6 SkillRegistry — decouple symbolic / reward / env / teacher (fixes #5)

```
@dataclass(frozen=True)
class Skill:
    name: str
```

```

operator: StructuredOperator          # §6.1
env_binding: EnvBinding               # env_id + kwargs + step budget
reward: RewardModule                 # §6.7
success_predicate: str
teacher: TeacherPolicy | None        # §6.10; None ⇒ held-out/novel
terminate_on_success: bool = True

class SkillRegistry:
    def register(self, skill: Skill) -> None: ...
    def get(self, name) -> Skill: ...
    def holdout(self, names) -> tuple[train_set, eval_set]: ... # for zero-shot splits

```

`holdout` makes "train on {A,B,C}, evaluate zero-shot on {D}" a first-class operation — the experiment rung 4 needs.

6.7 `RewardModule` + `RewardContext` (fixes #5, #6)

```

@dataclass(frozen=True)
class RewardContext:
    state: FetchState          # decoded once, shared
    facts: dict[str, bool]
    action: np.ndarray
    success: bool

@dataclass(frozen=True)
class RewardResult:
    reward: float
    components: dict[str, float]
    success: bool              # shaper may refine (putdown/stack "released")

class RewardModule(Protocol):
    def reset(self) -> None: ...
    def compute(self, ctx: RewardContext) -> RewardResult: ...
    # optional: declare return-scale so multi-skill training can normalize
    scale_hint: float

```

The `skill_reward` ladder collapses to `skill.reward.compute(ctx)`. A `RewardNormalizer` (using `scale_hint` or running stats) addresses the cross-skill magnitude confound (#6) explicitly.

6.8 `ObservationSchema` — named fields, no literal slices (fixes #7)

```

@dataclass(frozen=True)
class ObservationSchema:
    num_blocks: int
    def grip(self, raw) -> np.ndarray: ...
    def block_pos(self, raw, i) -> np.ndarray: ...
    def block_rot(self, raw, i) -> np.ndarray: ...
    def gripper_width(self, raw) -> float: ...
    def mover_base_indices(self, raw) -> tuple[int,int] | None: ...

```


`multiblock_env` and `FactEvaluator` both depend on **this**, so the producer and consumer of the layout can never silently diverge.

6.9 ExperimentConfig — one serializable spec (fixes #12)

A single dataclass (YAML/JSON-dumpable) that fully determines a run: `{skills, holdout, algo, timesteps, seed, pipeline_spec, conditioning_mode, teacher_spec, reward_norm, sampling}`. `naming.py` derives the filename *from* it; the config is written next to the model. Sweeps enumerate configs, not env vars.

6.10 TeacherPolicy + eval/interpretability (fixes #9, #10, #11)

```
class TeacherPolicy(Protocol):
    def action(self, obs, evaluator) -> np.ndarray: ... # None for novel skills

class EvaluationHarness:
    def success_rates(self, model, skills, *, episodes, seeds) -> dict[str, float]
    def zero_shot(self, model, held_out_skills, *, episodes) -> dict[str, float]

class InterpretabilityToolkit:
    def project(self, embeddings) -> np.ndarray # PCA/UMAP (have PCA)
    def component_ablation(self, op, pipeline, model, env) -> dict # Δsuccess per
    predicate/effect
    def arithmetic(self, ops, pipeline) -> ... # pickup-putdown analogies
    def interpolate(self, opA, opB, model, env) -> curve # success vs. blend
```

`component_ablation` — drop one predicate/effect from a `StructuredOperator`, recompose the embedding, measure the shared policy's Δ success — is the payoff of compositionality: it makes the embedding **interpretable at the symbol level**.

6.11 Current → target mapping

Today	Replaced/subsumed by
<code>SymbolicActionModel(description=string)</code>	<code>StructuredOperator</code> + <code>parse_pddl_operator</code> (§6.1)
<code>MockEmbedder</code> , <code>TextEmbedder</code>	<code>MockPipeline</code> , <code>WholeStringPipeline</code> (§6.4) — same iface as new <code>CompositionalPipeline</code>
<code>build_embedder</code> + trainable flag	<code>EmbeddingPipeline</code> (frozen <code>.as_numpy</code> vs learnable <code>nn.Module</code>) (§6.4)
<code>EnvWrapper</code> (obs-side)	<code>ConditioningModule mode="by_value"</code> (§6.5)
<code>SkillOneHotWrapper</code> + <code>SkillEmbeddingExtractor</code>	<code>ConditioningModule mode="table"/"compositional"</code> (§6.5)
<code>SKILLS</code> dict + <code>skill_reward</code> ladder	<code>SkillRegistry</code> + <code>RewardModule</code> (§6.6–6.7)

Today	Replaced/subsumed by
-------	----------------------

per-shaper <code>compute(**kwargs)</code>	<code>RewardModule.compute(RewardContext)</code> (§6.7)
slice indices in <code>facts.py</code>	<code>ObservationSchema</code> (§6.8)
argparse + env vars + filename	<code>ExperimentConfig</code> (§6.9)
<code>plot_embeddings.py</code>	<code>InterpretabilityToolkit</code> (superset) (§6.10)
<code>mock_env_embedder.py</code>	delete

7. Migration path (incremental, each phase shippable)

1. **Quick hygiene** 🟢 — delete `mock_env_embedder.py`; fix `EnvWrapper` to read the real obs shape (or route single-skill through `ConditioningModule`); document the openai-vs-hash semantic constraint.
2. **Structured operators** 🔴 — add §6.1 (`StructuredOperator` + parser), populate from existing `SKILLS` PDDL strings; keep old embedders working via `raw`. *No behavior change; unlocks everything else.*
3. **Unify embedding** 🔴 — introduce `EmbeddingPipeline` (§6.4); re-express Mock/Text as pipelines; retire the `trainable` double-meaning.
4. **Unify conditioning** 🔴 — `ConditioningModule` (§6.5) with `mode="table"` reproducing today's results (regression check), then add `by_value`.
5. **Compositional pipeline** 🔴 — `ComponentEmbedder` + `Aggregator` (§6.2–6.3); run the scheme ablation (mean/DeepSets/attention/GNN) vs mock/text baselines.
6. **Zero-shot** 🔴 — `SkillRegistry.holdout` + `ConditioningModule mode="compositional"`; measure held-out success with `EvaluationHarness`.
7. **Decouple rewards** 🔴 — `RewardModule/RewardContext` + normalizer; removes the ablation confound (#6). Can proceed in parallel with 2–6.
8. **Interpretability** 🟡 — `InterpretabilityToolkit` (component ablation, arithmetic, interpolation).
9. **Config + eval** 🟡 — `ExperimentConfig` + `EvaluationHarness` as the standard run/measure surface.

Phases 2 → 4 are the critical path; 5–6 deliver the thesis; 7 protects its validity; 8–9 make the results legible and reproducible.

8. Risk register (research-level, not code)

Risk	Nature	Mitigation hook in the design
Compositional generalization simply doesn't emerge	The real, unproven scientific claim	Aggregator is swappable (§6.3); baselines share the interface for honest comparison
Embeddings collapse (skills indistinguishable) under training	Trainable table/pipeline drift	<code>current_embeddings()</code> logging + <code>InterpretabilityToolkit.project</code> as a live monitor
Reward-scale confound masks the embedding effect	Methodological	<code>RewardNormalizer</code> + <code>scale_hint</code> (§6.7)
Zero-shot needs a teacher that novel skills lack	Structural	<code>teacher=None</code> marks held-out skills; eval path forbids BC on them (§6.6)
"Semantics helps" only measurable with paid API	Experimental cost	keep <code>hash</code> baseline; treat <code>openai</code> runs as the semantic arm, documented
Aggregator interpretability is opaque (esp. attention/GNN)	Rung 5	prefer <code>component_ablation</code> (behavioral, model-agnostic) as the primary probe

9. Appendix

Dead / stale: `symb_model_embeddings/mock_env_embedder.py` (torch import, skill names `pushdown/pushup` that don't exist in `SKILLS`; superseded by `MockEmbedder`).

Strong, keep-as-is: `backends.py`, `cache.py`, `multiblock_env.py`, `naming.py` — clean, encapsulated, reusable behind the new interfaces.

Highest-leverage single change: §6.5 `ConditioningModule`. It collapses two subsystems into one, and its `by_value/compositional` modes are the only thing that makes zero-shot (the project's headline goal) expressible at all.

Quick wins (< 1 day each): delete dead file; fix `EnvWrapper` shape; add the PDDL parser (strings already exist); write the current → target mapping table into the code as module docstrings so the migration is self-documenting.